pfi

Python for Fixed Income

COURSE 2 · BOND MATH

Yield From Price

WEEK 1 · DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's note: four quotes, no yields

The Desk Lead's Note

Dealers quote the four new bonds on price this morning: Quorvane 99.125, Dunmarrow 100.5, Pellwick 100.875, Halvering 97.75. Notebook 02 went from yield to price; I need the other way. What yield does each quote imply? Show me how Excel gets there and how close it lands. Put the search in "bondmath", docstring first, prove it against Excel's RATE and YIELD, and make it refuse a price no yield explains. If an edit breaks it, show me you can undo it, then take it home.

The issuers and quotes are invented; every number describes the data.

Four Bonds, Quoted on Price

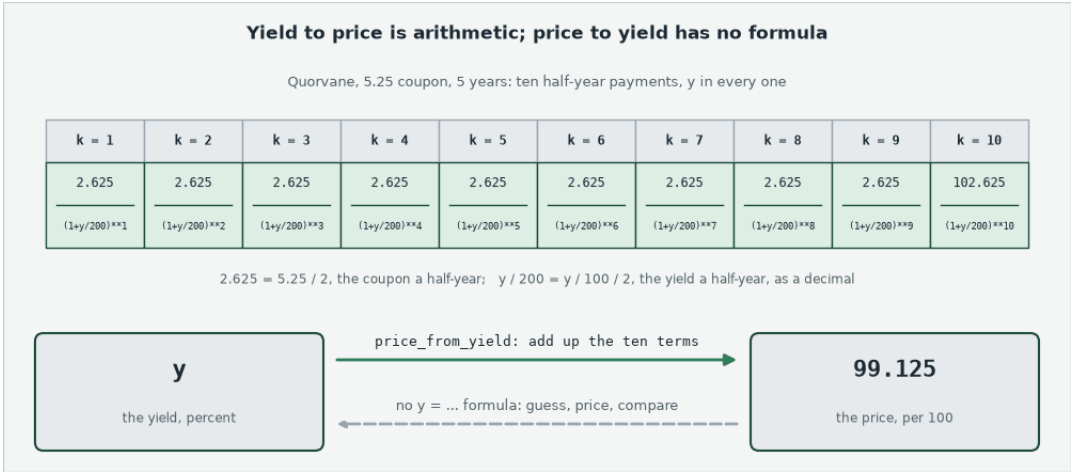
issuer	ticker	coupon_pct	years	quote
Quorvane Energy	QVNE	5.250	5	99.125
Dunmarrow Rail	DNMR	6.125	7	100.500
Pellwick Paper	PLWK	4.750	3	100.875
Halvering Foods	HLVF	7.500	10	97.750

```
........   [100%]
8 passed in 0.50s
```

No yield column: finding it is today's job. Dunmarrow and Pellwick are quoted above 100, so notebook 03's rule already says they yield less than their coupons.

Four prices, no yields: today the arrow turns.

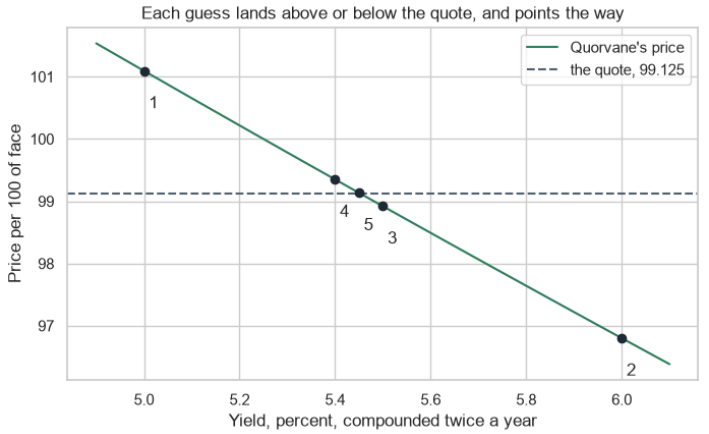
4.1 Why yield has no formula: guess, price, compare



Given a yield, the ten terms are arithmetic: notebook 02's "price_from_yield". Given the price, the yield sits inside all ten, each to a different power, and no algebra rearranges them into "y = ...". So a yield is found by search: guess a yield, price it, compare with the quote, adjust.

Every guess says which way to go, because price falls as yield rises: that one fact is all a search needs.

```
guesses = [5.0, 6.0, 5.5, 5.4, 5.45]
for guess_pct in guesses:
    guess_price = bondmath.price_from_yield(5.25, guess_pct, 5)
    side = 'above the quote, go higher' if guess_price > 99.125 else 'below the quote, go lower'
    print(f'{guess_pct:.2f} percent: price {guess_price:.4f}, {side}')
5.00 percent: price 101.0940, above the quote, go higher
6.00 percent: price 96.8012, below the quote, go lower
5.50 percent: price 98.9200, below the quote, go lower
5.40 percent: price 99.3503, above the quote, go higher
5.45 percent: price 99.1349, above the quote, go higher
```



Guesses 5 and 3, at 5.45 and 5.50, sit either side of the dashed line: the yield is where the line crosses the curve.

4.2 Goal Seek in Excel: close, and it depends on the start

Excel searches for you with Goal Seek: you name a cell holding a formula, the value it should show, and the cell Excel may change.

Excel side by side

Job	Excel	Result
A first guess	5 in A1	
The price at the guess	=PRICE(DATE(2026,9,30),DATE(2031,9,30),5.25/100,A1/100,100,2,0) in B1	
Guess until the price is the quote	Data, What-If Analysis, Goal Seek: Set cell B1, To value 99.125, By changing cell A1, then OK	A1 5.452248461259856, B1 99.12520671813279

Where Goal Seek Stopped, Started From 5

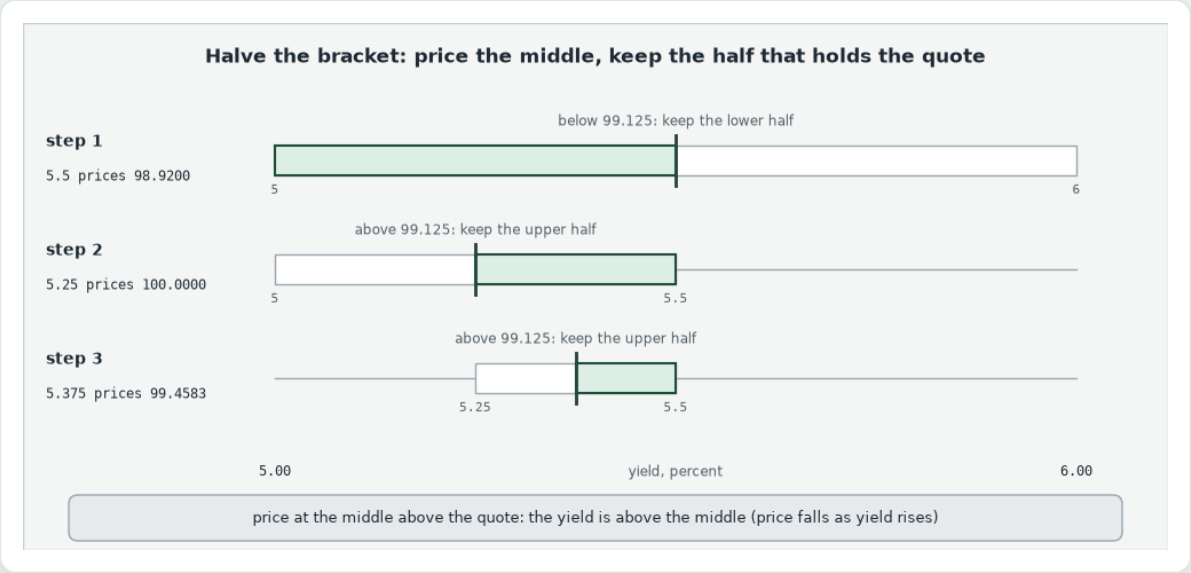
ticker	yield_pct	price	price_gap	excel_yield_pct	yield_gap_bp
QVNE	5.452248	99.125207	0.000207	5.452297	-0.004804
DNMR	6.036364	100.500001	0.000001	6.036364	-0.000015
PLWK	4.435154	100.875356	0.000356	4.435281	-0.012744
HLVF	7.828571	97.750262	0.000262	7.828610	-0.003889

Goal Seek lands close, not on the quote: Pellwick's price stops 0.000356 above it, and its yield 0.0127 bp from Excel's "YIELD". Start it from Quorvane's coupon, 5.25, and it stops somewhere else.

Job	Excel	Result
Start from the coupon	5.25 in A1, the same Goal Seek	A1 5.452293509043892

Where Goal Seek stops depends on where it starts: an answer key needs a search that stops where we say, every time.

4.3 Bisection: price the middle, keep the half



4.1's guesses put Quorvane's yield between 5 and 6 percent: call that the bracket. Now guess the same way every time: price the middle, and keep the half that holds the quote. Each step halves the bracket: bisection.

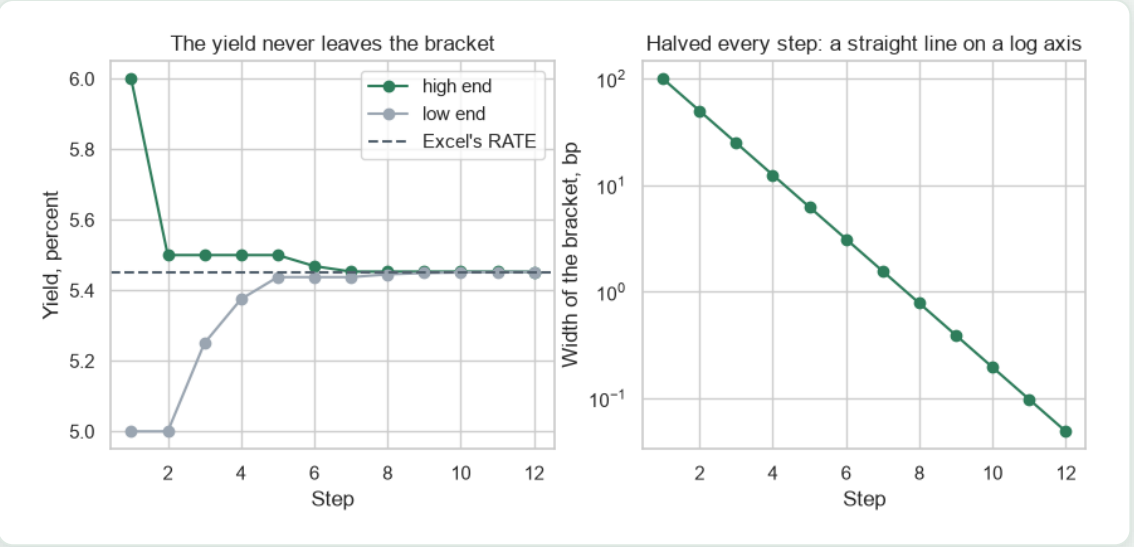
```
low_pct, high_pct = 5.0, 6.0
rows = []
for step in range(1, 13):
    mid_pct = (low_pct + high_pct) / 2
    price_at_mid = bondmath.price_from_yield(5.25, mid_pct, 5)
    rows.append({'step': step, 'low_pct': low_pct, 'high_pct': high_pct, 'mid_pct': mid_pct,
                 'price_at_mid': price_at_mid})
    if price_at_mid > 99.125:
        low_pct = mid_pct
    else:
        high_pct = mid_pct
halving = pd.DataFrame(rows)
halving['width_bp'] = (halving['high_pct'] - halving['low_pct']) * 100
```

Excel side by side: the same halving, one row per step

Job	Excel	Result
The bracket	low_pct, high_pct, mid_pct, price_at_mid in A1:D1, then 5 and 6 in A2:B2	
The middle, priced	=(A2+B2)/2 in C2, =PRICE(DATE(2026,9,30),DATE(2031,9,30),5.25/100,C2/100,100,2,0) in D2	
Keep a half	=IF(D2>99.125,C2,A2) in A3, =IF(D2>99.125,B2,C2) in B3, C2:D2 copied to row 3, then A3:D3 filled down to row 31	C13 5.452392578125, C31 5.452296503819525

Twelve halvings: the bracket closes

step	low_pct	high_pct	mid_pct	price_at_mid	width_bp
1	5.000000	6.000000	5.500000	98.9200	100.0000
2	5.000000	5.500000	5.250000	100.0000	50.0000
3	5.250000	5.500000	5.375000	99.4583	25.0000
4	5.375000	5.500000	5.437500	99.1887	12.5000
5	5.437500	5.500000	5.468750	99.0542	6.2500
6	5.437500	5.468750	5.453125	99.1214	3.1250
7	5.437500	5.453125	5.445312	99.1551	1.5625
8	5.445312	5.453125	5.449219	99.1382	0.7812
9	5.449219	5.453125	5.451172	99.1298	0.3906
10	5.451172	5.453125	5.452148	99.1256	0.1953
11	5.452148	5.453125	5.452637	99.1235	0.0977
12	5.452148	5.452637	5.452393	99.1246	0.0488



Steps 1 to 3 are the picture's. The width halves every row, from 100 bp to 0.0488; the twelfth middle is 5.452393.

The bracket halves every step and the yield never leaves it: bisection cannot miss.

4.4 A while loop that stops when narrow enough

Twelve steps came from "range(1, 13)". A better rule stops when the bracket is narrow enough, however many steps that takes: a "while" loop, from Course 1's Python for Fixed Income: Introduction, Part 2.

```
low_pct, high_pct = 5.0, 6.0
steps = 0
while high_pct - low_pct > 1e-10:
    mid_pct = (low_pct + high_pct) / 2
    if bondmath.price_from_yield(5.25, mid_pct, 5) > 99.125:
        low_pct = mid_pct
    else:
        high_pct = mid_pct
    steps += 1
quorvane_yield_pct = (low_pct + high_pct) / 2
print(f'{steps} steps: Quorvane at 99.125 yields {quorvane_yield_pct:.8f} percent')
34 steps: Quorvane at 99.125 yields 5.45229650 percent
```

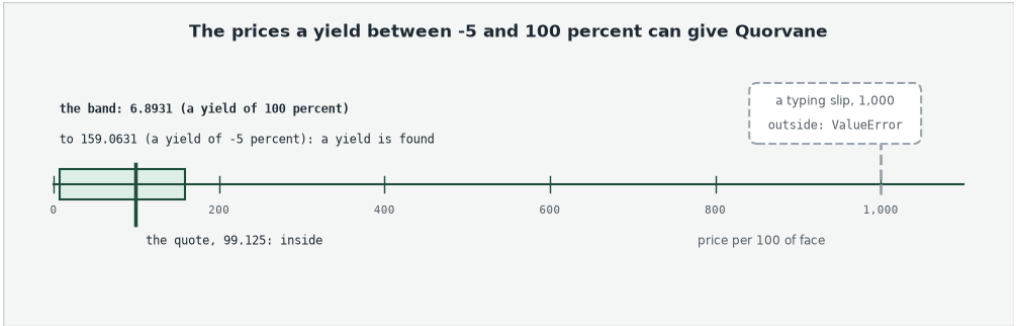
```
excel_rate_pct = tvm.loc['tvm_rate_11', 'excel_value'] * 100
gap_bp = abs(quorvane_yield_pct - excel_rate_pct) * 100
print(f"Excel's RATE: {excel_rate_pct:.8f} percent, a gap of {gap_bp:.1e} bp")
Excel's RATE: 5.45229650 percent, a gap of 2.0e-08 bp
```

Your loop lands on Excel's "RATE": 5.45229650 percent, 2.0e-08 bp apart.

4.5 What the Bracket Can Explain

Outside the bracket the loop still runs: it squeezes against one end and returns it, a wrong number with no warning.

```
highest_price = bondmath.price_from_yield(5.25, -5.0, 5)
lowest_price = bondmath.price_from_yield(5.25, 100.0, 5)
print(f'From {lowest_price:.4f} at 100 percent to {highest_price:.4f} at -5 percent')
From 6.8931 at 100 percent to 159.0631 at -5 percent
```



A price in the band has a yield in the bracket; a typing slip of 1,000 has none, and the function must say so.

4.5 yield_from_price, with a check that stops

```
%%writefile -a bondmath.py
def yield_from_price(coupon_pct: float, price: float, years: float, frequency: int = 2) -> float:
    low_pct, high_pct = -5.0, 100.0
    highest_price = price_from_yield(coupon_pct, low_pct, years, frequency)
    lowest_price = price_from_yield(coupon_pct, high_pct, years, frequency)
    if not lowest_price <= price <= highest_price:
        raise ValueError(f"price {price} is outside {lowest_price:.6f} to {highest_price:.6f}, "
                         f"the prices at yields of {high_pct} and {low_pct} percent")
    while high_pct - low_pct > 1e-10:
        mid_pct = (low_pct + high_pct) / 2
        if price_from_yield(coupon_pct, mid_pct, years, frequency) > price:
            low_pct = mid_pct
        else:
            high_pct = mid_pct
    return (low_pct + high_pct) / 2
```

Docstring first: the units, percent and per 100 in, percent out; the convention, compounded "frequency" times a year; the bracket and when the search stops; the "ValueError"; and Excel's route. Then the check: "lowest_price <= price <= highest_price" is a chained comparison, true only when both "<=" hold, and "not" turns it round, so a price outside raises. Then 4.4's loop.

A Price of 1,000

```
bondmath.yield_from_price(5.25, 1000.0, 5)
ValueError: price 1000.0 is outside 6.893110 to 159.063092,
  the prices at yields of 100.0 and -5.0 percent
```

Four quotes, four yields, against RATE and YIELD

ticker	coupon_pct	quote	yield_pct	excel_rate_pct	excel_yield_pct
QVNE	5.250	99.125	5.452297	5.452297	5.452297
DNMR	6.125	100.500	6.036364	6.036364	6.036364
PLWK	4.750	100.875	4.435281	4.435281	4.435281
HLVF	7.500	97.750	7.828610	7.828610	7.828610

Three columns, one answer to the sixth decimal.

Notebook 03's rule, read from the other side: Dunmarrow and Pellwick yield less than their coupons, Quorvane and Halvering more.

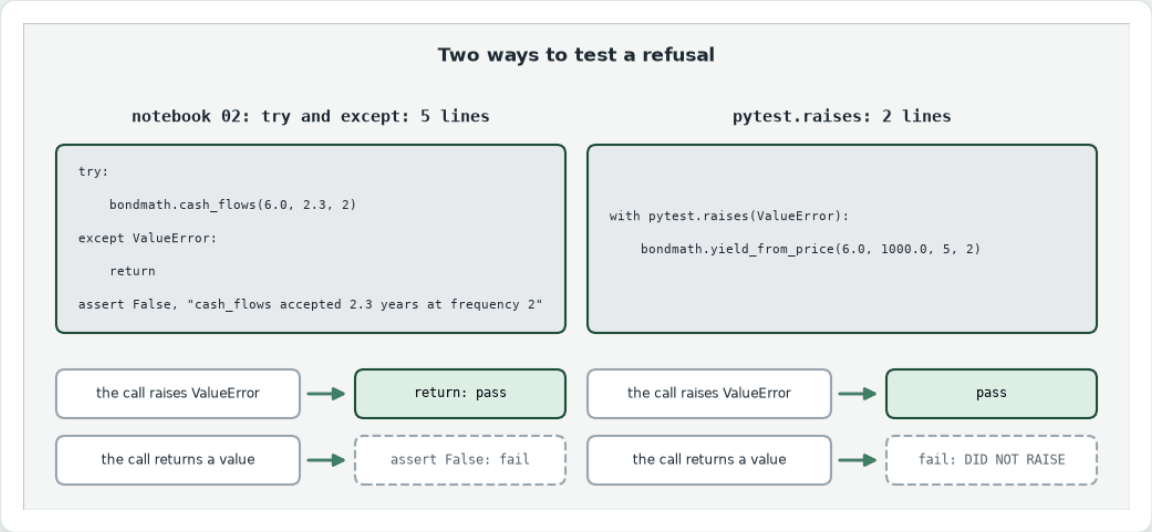
Excel side by side. "RATE(nper, pmt, pv, fv)" takes the periods, the coupon a period, the price with a minus sign (money out, as notebook 01's "PV"), and the 100 back, and returns a rate a period, so "*2" makes it annual.

Job	Excel	Result
Yield from price, from periods	`=RATE(2*5,5.25/2,-99.125,100)*2`	0.05452296503527843
Yield from price, from dates	`=YIELD(DATE(2026,9,30),DATE(2031,9,30),5.25/100,99.125,100,2,0)`	0.054522965032957
A price of 1,000	`=YIELD(DATE(2026,9,30),DATE(2031,9,30),5.25/100,1000,100,2,0)*100`	-39.3451
A price of 0	`=ISERROR(YIELD(DATE(2026,9,30),DATE(2031,9,30),5.25/100,0,100,2,0))`	TRUE

"YIELD" searches past -5 percent; the course's bracket is a desk's choice, treating a price that far from par as a typing slip.

Four quotes, four yields that match Excel, and a price no yield in the bracket explains is refused.

4.6 pytest.raises: a test that passes only on a refusal



The first reads Excel's 18 "RATE" rows; each row's formula multiplies "RATE" by the frequency, so its value is the annual yield as a decimal, and the expected value is that times 100. It then keeps the coupon-date rows with notebook 02's filter; their "yield_pct_from_price" is Excel's "YIELD", already in percent. The second tests one price too high for any yield in the bracket and one too low.

The refusal needs a test that passes only if the function raises. "pytest.raises" says it in a "with" block, and "as caught" keeps the error: "caught.value" is the error itself.

```
import pytest
with pytest.raises(ValueError) as caught:
    bondmath.yield_from_price(5.25, 1000.0, 5)
print('Caught:', caught.value)
Caught: price 1000.0 is outside 6.893110 to 159.063092,
  the prices at yields of 100.0 and -5.0 percent
```

When Nothing Is Raised

```
with pytest.raises(ValueError):
    bondmath.yield_from_price(5.25, 99.125, 5)
Failed: DID NOT RAISE ValueError
```

Two tests: Excel's rows, and the refusal

```
%%writefile -a test_bondmath.py
def test_yield_from_price_matches_excel():
    for row in tvm_rows("RATE"):
        result = bondmath.yield_from_price(float(row["coupon_pct"]), float(row["price"]),
                                           float(row["years"]), int(row["frequency"]))
        assert result == pytest.approx(float(row["excel_value"]) * 100, abs=1e-6), row["case_id"]
    coupon_date_rows = [row for row in REFERENCE if row["coupdaybs"] == "0"]
    assert coupon_date_rows, "no coupon-date rows in the reference file"
    for row in coupon_date_rows:
        frequency = int(row["frequency"])
        years = int(row["coupnum"]) / frequency
        result = bondmath.yield_from_price(float(row["coupon_pct"]), float(row["price"]), years, frequency)
        assert result == pytest.approx(float(row["yield_pct_from_price"]), abs=1e-6), row["case_id"]
def test_yield_raises_outside_bracket():
    with pytest.raises(ValueError):
        bondmath.yield_from_price(6.0, 1000.0, 5, 2)
    with pytest.raises(ValueError):
        bondmath.yield_from_price(6.0, 0.01, 5, 2)
```

```
.........   [100%]
10 passed in 0.50s
```

How Many Excel Numbers

```
18 RATE rows and 17 coupon-date YIELD rows
Frequencies [1, 2, 4]
```

4.7 Price to yield to price: the round trip

Two functions that undo each other can check each other. Q. Price each bond at its issue yield from notebook 02, then solve the yield back.

```
trip = bonds[['ticker', 'coupon_pct', 'years']].copy()
trip['issue_yield_pct'] = [5.4, 6.125, 4.6, 7.7]
terms = list(zip(trip['coupon_pct'], trip['issue_yield_pct'], trip['years']))
trip['price'] = [bondmath.price_from_yield(c, y, t) for c, y, t in terms]
trip['yield_back_pct'] = [bondmath.yield_from_price(c, p, t)
                          for c, p, t in zip(trip['coupon_pct'], trip['price'], trip['years'])]
trip['gap_bp'] = (trip['yield_back_pct'] - trip['issue_yield_pct']) * 100
formats = {'issue_yield_pct': '{:.3f}', 'price': '{:.6f}', 'yield_back_pct': '{:.9f}',
           'gap_bp': '{:+.1e}'}
trip[['ticker', 'issue_yield_pct', 'price', 'yield_back_pct', 'gap_bp']].style.format(
    formats).hide(axis='index')
```

ticker	issue_yield_pct	price	yield_back_pct	gap_bp
QVNE	5.400	99.350327	5.400000000	+1.1e-09
DNMR	6.125	100.000000	6.125000000	-1.9e-09
PLWK	4.600	100.415887	4.600000000	-1.6e-09
HLVF	7.700	98.622736	7.700000000	+1.6e-09

Every yield comes back within 1.9e-09 bp and every quote within 2.5e-10 per 100; Goal Seek stopped up to 0.0004 away. The round trip proves the two functions agree with each other; Excel's rows prove they are right.

```
%%writefile -a test_bondmath.py
def test_yield_round_trip():
    bond_price = bondmath.price_from_yield(5.25, 6.1, 7, 2)
    assert bondmath.yield_from_price(5.25, bond_price, 7, 2) == pytest.approx(6.1, abs=1e-8)
```

Quote, Yield, Quote

ticker	quote	price_back	gap
QVNE	99.125	99.125000000	+1.4e-11
DNMR	100.500	100.500000000	+2.5e-10
PLWK	100.875	100.875000000	+5.2e-11
HLVF	97.750	97.750000000	-1.2e-10

```
..........   [100%]
11 passed in 0.50s
```

Two functions that undo each other to nine decimals: the round trip proves they agree, Excel's rows that they are right.

4.8 The undo button: commit first, then a bad edit

A commit is a saved copy you can go back to, so commit first.

```
!git -C "{work}" status --short
!git -C "{work}" diff --stat
 M bondmath.py
 M test_bondmath.py
?? __pycache__/
 bondmath.py    | 29 ++++++++++++++++++++++++++++
 test_bondmath.py | 30 ++++++++++++++++++++++++++++++
 2 files changed, 59 insertions(+)
```

" M" marks a file modified since the last commit. "??" marks one Git does not track: "__pycache__", the compiled copies Python keeps, which notebook 06 tells Git to ignore.

```
subject = 'Notebook 04: yield_from_price'
body = 'Desks quote on price; the yield makes two quotes comparable'
!git -C "{work}" add bondmath.py test_bondmath.py
!git -C "{work}" commit -m "{subject}" -m "{body}"
!git -C "{work}" log --oneline
[main 07294a2] Notebook 04: yield_from_price
 2 files changed, 59 insertions(+)
07294a2 Notebook 04: yield_from_price
3f57cac Notebook 03: convert_yield
408a678 Notebook 02: cash_flows and price_from_yield
d029474 Notebook 01: future_value and present_value
```

Suppose an edit, yours or an assistant's, tidies "yield_from_price" by deleting the check: no real bond yields outside -5 to 100 percent. A string's ".index()" gives the position where a piece of text starts, as a list's ".index()" gave an item's in Course 1's Case Study: Organizing a Small Portfolio.

```
with open('bondmath.py', encoding='utf-8') as file:
    source = file.read()
start = source.index('    highest_price')
end = source.index('    # halve the bracket')
with open('bondmath.py', 'w', encoding='utf-8') as file:
    file.write(source[:start] + source[end:])
importlib.reload(bondmath)
print(f'A price of 1,000 now gets {bondmath.yield_from_price(5.25, 1000.0, 5):.4f} percent')
A price of 1,000 now gets -5.0000 percent
```

No error: -5.0000, the bottom of the bracket, the silent wrong number 4.5 warned of.

Read the diff, then put it back

```
git diff bondmath.py      @@ -124,11 +124,6 @@: from line 124, 11 lines became 6

-    highest_price = price_from_yield(coupon_pct, low_pct, years, frequency)                    5 lines with - removed
-    lowest_price = price_from_yield(coupon_pct, high_pct, years, frequency)                    no line with +
-    if not lowest_price <= price <= highest_price:
-        raise ValueError(f"price {price} is outside {lowest_price:.6f} to {highest_price:.6f}, "
-                         f"the prices at yields of {high_pct} and {low_pct} percent")
```

Five "-" lines and no "+": the diff names exactly what the edit took.

```
!git -C "{work}" restore bondmath.py
!git -C "{work}" status --short
?? __pycache__/
```

No " M" beside "bondmath.py": it is the committed file again. "git restore" throws the change away for good, so read the diff first.

Excel side by side. The nearest move in a workbook is to close it without saving: you are back at the last save.

The module in memory still holds the edit, so reload it from the restored file first.

```
importlib.reload(bondmath)
with pytest.raises(ValueError) as caught:
    bondmath.yield_from_price(5.25, 1000.0, 5)
print('Refused again:', caught.value)
Refused again: price 1000.0 is outside 6.893110 to 159.063092,
  the prices at yields of 100.0 and -5.0 percent
```

```
..........   [100%]
11 passed in 0.50s
```

Bring it home, and what you can now do

As in notebook 03, in a PowerShell terminal with the course's ".venv" active, with your own work folder. The copy replaces your two files: anything you added by hand shows in "git diff --stat" as "-" lines, to read before you commit.

```
$work = "$env:TEMP\pfi_course2_04_k4w7q2xr"
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$work\bondmath.py", "$work\test_bondmath.py" .
python -m pytest -q
git diff --stat
git add bondmath.py test_bondmath.py
git commit -m "Notebook 04: yield_from_price" -m "Desks quote on price; the yield makes two quotes comparable"
git log --oneline
```

The undo button works the same at home, before you commit:

```
git status --short
git diff bondmath.py
git restore bondmath.py
```

In Colab, download the two ".py" files from the file browser on the left. The work folder can stay: a restart makes a new one.

"my_bondmath" holds six functions, eleven tests, and four commits.

You Can Now

Say why a yield has no formula, and why price falling as yield rises is all a search needs.

Find a yield with Excel's Goal Seek, and say why its answer depends on where it starts.

Halve a bracket by hand in Python and in a sheet, and say why bisection cannot miss.

Write a "while" loop that stops when the bracket is narrow enough.

Solve a yield from a price with "yield_from_price", and in Excel with "RATE" and "YIELD".

Test a refusal with "pytest.raises", and two functions against each other with a round trip.

Undo a bad edit with "git status", "git diff", and "git restore".

Predict first: a wider bracket, and paid 130

Predict 1

Start from -5 to 100 percent instead: a bracket 105 times wider, wide enough for any bond in the course. How many halvings to get below 1e-10 percent? Write down a number.

```
low_pct, high_pct = -5.0, 100.0
steps = 0
while high_pct - low_pct > 1e-10:
    mid_pct = (low_pct + high_pct) / 2
    if bondmath.price_from_yield(5.25, mid_pct, 5) > 99.125:
        low_pct = mid_pct
    else:
        high_pct = mid_pct
    steps += 1
print(f'{steps} steps: {(low_pct + high_pct) / 2:.8f} percent')
```

Predict 2

Quorvane pays 126.25 in all, per 100 of face. A buyer pays 130.00 for it. What yield does the function return?

A) about -0.6 percent, the 3.75 lost spread over five years

B) about -2.9 percent, the 3.75 lost as a share of 130.00

C) about -3.8 percent, the 3.75 lost read as a percent

```
print(f'Quorvane at 130.00: {bondmath.yield_from_price(5.25, 130.0, 5):.4f} percent')
```

Predict first: paid 50, and the bad edit

Predict 3

Halve the price: a buyer pays 50.00 for Quorvane. Near 99 it yields about 5.45 percent. At 50?

A) about double, near 11 percent

B) far more than double

C) less than double, because the coupon stays the same

```
print(f'Quorvane at 50.00: {bondmath.yield_from_price(5.25, 50.0, 5):.4f} percent')
```

Predict 4

Run the 11 tests on the edited file. How many fail?

A) none, the function still runs

B) one, "test_yield_raises_outside_bracket"

C) all three of today's

```
import subprocess
import sys
command = [sys.executable, '-m', 'pytest', '-q', '-p', 'no:cacheprovider', '--tb=no']
result = subprocess.run(command, cwd=work, capture_output=True, text=True)
print(result.stdout + result.stderr)
```

The note, answered

The note asked	Answer
What each quote yields	Quorvane 5.4523, Dunmarrow 6.0364, Pellwick 4.4353, Halvering 7.8286 percent
How Excel gets there	Goal Seek, up to 0.0004 from the quote; RATE and YIELD, to the sixth decimal
The search in the module	yield_from_price, 40 halvings from -5 to 100 percent
Proved against Excel	11 passed, 35 Excel rows within 1e-6 percent
A price no yield explains	a ValueError, pinned by pytest.raises
An edit that breaks it	caught by a test, read in the diff, undone by git restore
Taken home	my_bondmath, four commits

Be proud: you solved for a number that has no formula and proved it against Excel to a millionth of a percent.

Next, Case Study: A New Issue Calendar (notebook 05): eight re-offer prices turned into yields, and a workbook Excel calculates itself.